Exercise Sheet 5 – Centralized Active Replication Protocol and Synchronization

Lab on 17.12.25 and 19.12.25

In this lab you will implement two consistency protocols from the lecture for our message board application. It is the Centralized Active Replication Protocol and a synchronization scheme inspired by Bayou.

You have to submit your solutions by showing them to your tutor. The latest submission is possible in the lab, in which the next exercise sheet is presented (see table below). You can also submit in the lab with the question time on the exercise sheet.

	Presentation of the tasks	Question time	Last possible submission
Group 1:	Wednesday, 17.12.25	Wednesday, 7.1.26	Wednesday, 14.1.26
Group 2:	Friday, 19.12.25	Friday, 9.1.26	Friday, 16.1.26

For the tasks of this exercise sheet you can get 8 points in total. You get it for:

	Task 1a) + b)	Task 1c)	Task 2	Task 3a) + b)	Task 3c)	Task 3d) + e)
Points:	1	2	1	1	2	1

Task 1 Centralized Active Replication Protocol

Implement the Centralized Active Replication Protocol (lecture slides 07: Consistency & Replication III) for forwarding updates in our message board application. The idea is as follows: Updates are performed by all servers in the same order to ensure consistency. To do so, when a server receives an update operation from a client, it adds a sequence number to the update operation. The sequence number is queried from a central server called sequencer (the coordinator). Thus, update operations have unique, increasing sequence numbers (1, 2, 3, …) over all servers. The update operation including its sequence number is forwarded to all other servers. All servers perform the update operations in the order of the sequence numbers. If a sequence number is missing, a server waits. It then does not execute an update operation with a higher sequence number. In your solution, ignore issues with fault tolerance. E.g., updates become impossible, if a server queries a sequence number and then crashes, before forwarding the update operation to all other servers.

a) Implement the sequencer and include it into `AsyncBoardProxy` and the stub.

   Start with the module `Sequencer.py` given in file `DSLabs5_GivenFiles.zip`, which you can download from StudIP. Unzip the file and put all contained files in your server or client directory with the results from the previous labs. `Sequencer.py` contains the class `sequencer` that only has a constructor und one method `getSequenceNumber()`. The method returns a sequence number 1, 2, 3, 4, … Its first call returns 1, its second call 2, and so on.

   Add the method `getSequenceNumber()` to `AsyncBoardProxy` and the stub. The sequencer object is given to your function `startServer()` in module `BoardServer.py` as additional parameter `sequencerParam`. The parameter has a default value of `None`. If

it is `None` and the remote function `getSequenceNumber()` is called, then the stub returns an error.

Test your sequencer by calling it remotely via the proxy. Start AsyncIO REPL and create a proxy object from `AsyncBoardProxy`. Call the remote function `getSequenceNumber()` several times and check its results.

For the test you need to start a server that contains the sequencer. Implement a module `CentralizedActiveReplicationProtocol_Main.py` that creates the required objects and starts your server. Copy file `UseMutexForUpdates_Main.py` into the new file and modify it as required. For the tests in this subtask, you need to create the sequencer object and pass it as parameter to `BoardServer.startServer()`. Delete the creation of the mutex that is not needed for this task.

b) Add an additional parameter for a sequence number to all update operations in `AsyncBoardProxy` and the stub.

If a server calls an update method (`put()`, `modify()`, `delete()`, `deleteAll()`) on another server, it needs to be able to include the sequence number. Implement that with an additional, optional parameter `sequenceNumber` for each update method. It should have a default value of `None`, if no sequence number is included. Implement your code, so that a value of `None` does not lead to an error to support compatibility with other message board versions.

Implement the additional parameter `sequenceNumber` for each update operation in `AsyncBoardProxy` and in the stub. In principle, you have to add the parameter also to all classes with update methods that are called by the stub. As the stub will now provide this parameter, you would get an error otherwise. It is all classes for distributing messages to other servers that you have written in previous labs as well as `AsyncBoardStorage.py`. You need to add this parameter at least to `UseMutexForUpdates.py` for testing in this task, and to modules implemented in future tasks. Alternatively, you can try to only pass the parameter if the sequence number is not `None`.

To test your code, call update methods from `AsyncBoardProxy` using AsyncIO REPL in a server using a mutex (`UseMutexForUpdates.py`). Add debugging output to the methods `put()`, `modify()`, `delete()`, `deleteAll()` in `UseMutexForUpdates.py` that displays the sequence number. Start a server with `CentralizedActiveReplicationProtocol_Main.py` from the previous subtask. Call the update methods in the proxy with and without sequence number and observe if it is displayed on the server.

c) Implement the Centralized Active Replication Protocol for our message board system.

The server must execute update operations in the order of their sequence numbers. It must wait, if it has not received the update operation with the next sequence number, even if it already received update operations with higher sequence numbers. To implement this, all update operations are executed by an additional coroutine. When `put()`, `modify()`, `delete()`, or `deleteAll()` is called, the methods put the desired update operation in a queue, that is read and executed by the coroutine. A specific queue (`asyncio.PriorityQueue`) is used that automatically sorts update operations by their sequence numbers.

Class `asyncio.PriorityQueue` has the same methods as `asyncio.Queue`, which we have used in lab 3 in `UseMutexForUpdates.py`. However, `asyncio.PriorityQueue` sorts the elements that were added with `put()`. When `get()` is called, the element with the lowest value is returned (and removed). To sort update operations by their sequence number, you need to add tuples or lists with the sequence number as the element. E.g. you may add a tuple `(5, "PUT", "Hallo")` with sequence number 5 to perform a `put()` operation for the message "Hallo".

The methods `put()`, `modify()`, `delete()`, and `deleteAll()` check if calls came from a client or from another server. If it came from a client, they retrieve the next sequence number from the sequencer and forward the call with sequence number to all other servers. In both cases they put the update operation into the queue.

The coroutine gets the next element from the queue. It checks if the update operation has the expected next sequence number. If not, it puts the update operation back to the queue and waits some time. If it has the expected sequence number, the operation is performed on the local copy of the message board.

It is recommended to start with the code from `UseMutexForUpdates.py`. Even though that algorithm follows different concepts, it contains a lot of code needed for this algorithm (e.g. queue, additional coroutine). Copy the file `UseMutexForUpdates.py` into the new file `CentralizedActiveReplicationProtocol.py` in which you implement the new algorithm. Do not forget to update `CentralizedActiveReplicationProtocol_Main.py` to create an object from `CentralizedActiveReplicationProtocol.py` instead of from `UseMutexForUpdates.py`.

Test your code. Start a cluster consisting of four servers and test it with `UserInterface.py`.

Task 2 Measurements and Experiments

Repeat the experiments from the previous exercise sheets for your cluster with the Centralized Active Replication Protocol.

a) Use the script `SendDataToSeveralServers.py` to upload 4 messages concurrently to each server and observe their message boards. Are there the same 16 messages on each server? Are these in the same order? If not, your program from Task 1 has an error. (Correct it!) Take notes about the results you have observed.

b) Use the script `SendDataToSeveralServers.py` to measure the uploading time when concurrently uploading 1000 messages to each of the four servers. As `CentralizedActiveReplicationProtocol.py` uses a queue for buffering update operations, the execution time shown by the program might be too short. Use a stop watch (e.g. of your smartphone) to measure the time until the logging activity on your servers stops. Compare your results to those of the previous exercises. Write down the time for uploading the messages and your observations.

Task 3 Synchronization

Implement synchronization for our message board application, in which servers work mostly independent and pairs of two servers can be synchronized with each other. Update operations are not immediately called in other servers. Instead, clients must explicitly trigger the synchronization of two servers. This is inspired by Bayou explained in the lecture. However, as there are no conflicts between messages, no conflicts have to be resolved.

After synchronization, the servers must return the same messages in the same order. To do so, we reuse our vectors clock and sorting of messages from lab 4.

a) Create servers with a vector clock and sorting without continuous synchronization.

To do so, reuse the code from `InformAllOtherServersWithClock.py` written in lab 4. It already contains the vector clock and sorting. Copy it into a new file `Synchronize.py` and change that file. In each update operation (`put()`, `modify()`, `delete()`, and `deleteAll()`), remove the code for calling the same operation in the other servers. The code adds a timestamp to each new message from a client. For later synchronization, keep the code that prevents adding a timestamp if a call comes from another server. Messages are already sorted by the time stamps in module `AsyncBoardStorage.py`.

You need a module `Synchronize_Main.py` that creates and connects the objects needed by a server. Copy file `InformAllOtherServersWithClock_Main.py` into `Synchronize_Main.py` and update it as required. It needs to create an object from `Synchronize.py` instead of from `InformAllOtherServersWithClock.py`.

To test your code, start a cluster consisting of the usual four servers. Use the client program `SendDataToSeveralServersWithSynchronisation.py` to upload 4 messages to each server. The program was given in `DSLabs5_GivenFiles.zip` that you had already downloaded for Task 1. It is mostly the same as `SendDataToSeveralServers.py`, but you will extend it in the subtasks c) and d). In the test results, check that each server only contains the messages directly uploaded to it! (Otherwise there is still code forwarding `put()` to other servers.) What do you notice about the timestamps?

b) Add an empty remote procedure `synchronize()` to your server and to the client proxy.

The goal is to allow that a client can start the synchronization between two servers. For that purpose you implement the remote procedure `synchronize()` as additional method in `Synchronize.py`. It is called in one of the servers to be synchronized. The ID of the other server is given to the method as parameter. To allow a client to call the method, you need to add it to the stub in `BoardServer.py` and to `BoardProxy.py` of the client.

Test your code by calling the remote procedure `synchronize()` from a client. To do so, add debugging output to the procedure (using `print()`) that also shows its parameter. Add code to the client `SendDataToSeveralServersWithSynchronisation.py` that calls `synchronize()` for synchronizing the two servers with IDs 0 and 1. Add this call at the comment "`Add you code for synchronization of servers here.`" Start `SendDataToSeveralServersWithSynchronisation.py` and observe if your debugging output is shown in the terminal window of the correct server.

c) Implement the method `synchronize()` to synchronize two servers.

The idea is, to check for all messages in both servers, if a message with the same timestamp is also available on the other server. If not, it is created on the other server with `put()`. Before that, both message boards are queried from the servers by calling `getBoard()`. For the local server `getBoard()` called on its local storage (`AsyncBoardStorage.py`). For the other server it is called via its proxy.

There are semantic issues for the synchronization when calling `modify()`, `delete()`, and `deleteAll()`. Think about what these are. Ignore them in your implementation.

To test your code, start a cluster with the usual four servers. Start the program `SendDataToSeveralServersWithSynchronisation.py` from b) for uploading 4 messages to each server. After upload it synchronizes the servers with IDs 0 and 1. Do these two servers contain the same messages in the same order after synchronization? Do the other two servers still only contain the messages that were directly uploaded to it?

d) Implement the synchronization of all servers on the client.

Add code to `SendDataToSeveralServersWithSynchronisation.py` that synchronizes the four servers with each other. After synchronization, all servers should contain all messages uploaded to one of the severs in the same order. Which pairs of two servers have to be synchronized with each other? Start the program for uploading 4 messages to each server. Do all servers contain the same 16 messages in the same order?

e) Use the script `SendDataToSeveralServersWithSynchronisation.py` for uploading 1000 messages to each of the four servers. The program shows the times for uploading and synchronizing servers separately. Compare these times with the upload times from the previous exercises. Write down the times and your observations.